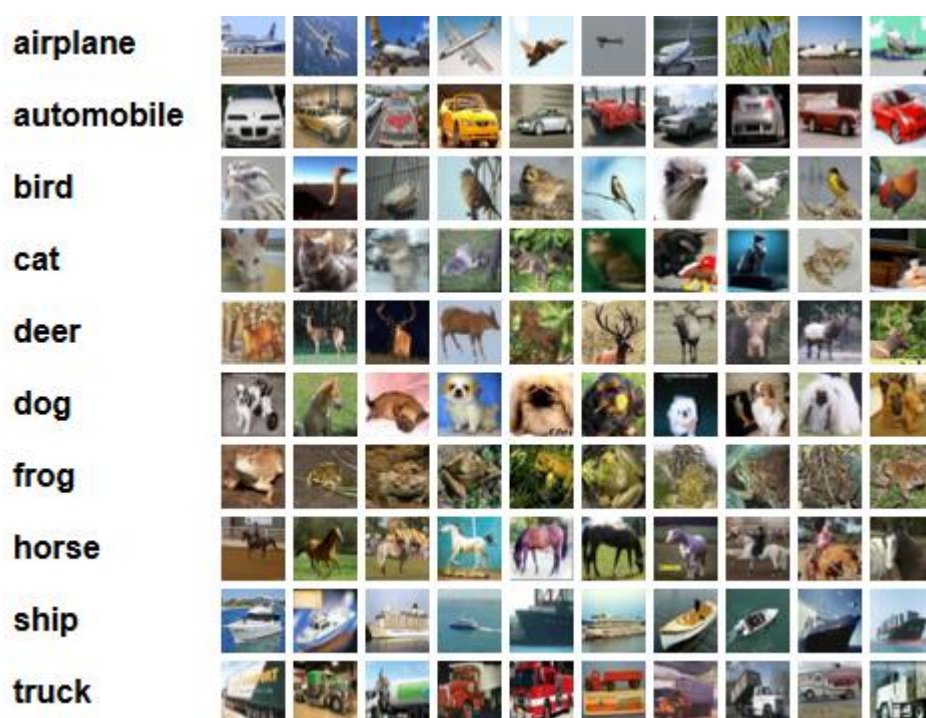


Building a CNN on CIFAR-10 with PyTorch

Dataset and Preprocessing

Dataset Overview:

CIFAR-10 影像分類資料集。由 60,000 張 32x32 像素的彩色圖片 (RGB) 組成，涵蓋 10 個不同的類別（如飛機、汽車、鳥類等）。資料集已劃分為 50,000 張的 Training Set 與 10,000 張的測試集 Test Set。



Data Preprocessing & Augmentation:

Data Augmentation - 僅限訓練集:

`RandomCrop(32, padding=4)`：在影像四周補上 4 個像素的零填充，再隨機裁切回原始的 32x32 大小。賦予模型平移不變性。

`RandomHorizontalFlip()`：隨機將影像進行水平翻轉，增加訓練樣本的多樣性。

Tensor Conversion:

`ToTensor()`：將原始影像轉換為 PyTorch 的 Tensor 格式，同時將像素值從 [0, 255] 縮放至 [0.0, 1.0] 的浮點數範圍。

Normalization - 訓練集與測試集皆使用:

Normalize(mean=(0.4914, 0.4822, 0.4465), std=(0.2470, 0.2435, 0.2616)) : 使用 CIFAR-10 預先計算好的全局 RGB 均值與標準差對資料進行標準化。加快梯度下降的收斂速度並提高穩定性。

Architecture Design and Shape Analysis

Layer-by-Layer Shape Trace:

```
# Input shape: (B, 3, 32, 32)
# Output: (B, 32, 32, 32)
self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
self.relu1 = nn.ReLU()
# Output: (B, 32, 16, 16)
self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

# Output: (B, 64, 16, 16)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
self.relu2 = nn.ReLU()
# Output: (B, 64, 8, 8)
self.pool2 = nn.MaxPool2d(kernel_size=2, stride=2)

# Output: (B, 128, 8, 8)
self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.relu3 = nn.ReLU()

# Output: (B, 128*8*8) = (B, 8192)
self.flatten = nn.Flatten()
# Output: (B, 512)
self.fc1 = nn.Linear(128 * 8 * 8, 512)
self.relu4 = nn.ReLU()
# Output: (B, 10)
self.fc2 = nn.Linear(512, 10)
```

Performance Analysis:

Parameter Count

$$\text{Conv1: } (3 \times 3 \times 3 + 1) \times 32 = 28 \times 32 = 896$$

$$\text{Conv2: } (32 \times 3 \times 3 + 1) \times 64 = 289 \times 64 = 18496$$

$$\text{Conv3: } (64 \times 3 \times 3 + 1) \times 128 = 577 \times 128 = 73856$$

$$\text{FC1: } (8192 + 1) \times 512 = 4194816 \text{ (佔據全模型 98\% 的參數)}$$

$$\text{FC2: } (512 + 1) \times 10 = 5130$$

$$\text{Total Parameters: } 896 + 18496 + 73856 + 4194816 + 5130 = \mathbf{4293194}$$

FLOPs Analysis

$$\text{Conv1: } 3 \times 3 \times 3 \times 32 \times 32 \times 32 = 884736$$

$$\text{Conv2: } 32 \times 3 \times 3 \times 64 \times 16 \times 16 = 4718592 \text{ (卷積層主導運算時間)}$$

$$\text{Conv3: } 64 \times 3 \times 3 \times 128 \times 8 \times 8 = 4718592$$

$$\text{FC1: } 8192 \times 512 = 4194304$$

$$\text{FC2: } 512 \times 10 = 5120$$

$$\text{FLOPs: } 884\text{K} + 4.71\text{M} + 4.71\text{M} + 4.19\text{M} + 5\text{K} \approx \mathbf{14521344}$$

```
=====
Performance Analysis: Basic CNN
=====
```

```
1. Parameters: 4.2932 M
2. Model Size: 16.38 MB
3. FLOPs:      0.014579 GFLOPs
4. Latency:    0.41 ms (avg over 100 runs)
5. Peak Mem:   24.86 MB (batch_size=1 inference)
=====
```

```
=====
Performance Analysis: ResNet CNN
=====
```

```
1. Parameters: 0.2901 M
2. Model Size: 1.11 MB
3. FLOPs:      0.030247 GFLOPs
4. Latency:    0.83 ms (avg over 100 runs)
5. Peak Mem:   51.08 MB (batch_size=1 inference)
=====
```

Receptive Field:

Receptive Field, R 代表當前層的單一神經元在原始輸入影像上所能看見的像素範圍。 J 代表累積的特徵圖 Stride。

Input:

$$R_0 = 1, J_0 = 1$$

Conv1 ($k = 3, s = 1$):

$$R_1 = 1 + (3 - 1) \times 1 = 3 \quad (J_1 = 1)$$

MaxPool1 ($k = 2, s = 2$):

$$R_2 = 3 + (2 - 1) \times 1 = 4 \quad (J_2 = 1 \times 2 = 2)$$

Conv2 ($k = 3, s = 1$):

$$R_3 = 4 + (3 - 1) \times 2 = 8 \quad (J_3 = 2)$$

MaxPool2 ($k = 2, s = 2$):

$$R_4 = 8 + (2 - 1) \times 2 = 10 \quad (J_4 = 2 \times 2 = 4)$$

Conv3 ($k = 3, s = 1$):

$$R_5 = 10 + (3 - 1) \times 4 = 18$$

Implementation Walkthrough

Training Loop

設定 `model.train()`，並在每個 Batch 實作五步驟訓練迴圈：

Zero Gradients (梯度歸零): `optimizer.zero_grad()`，在每次反向傳播前清除上一步的梯度累積，防止梯度計算錯誤。

Forward Pass (前向傳播): `outputs = model(inputs)`，將影像輸入模型，計算出各類別的預測 logits。

Compute Loss (計算損失): `loss = criterion(outputs, labels)`，使用 `nn.CrossEntropyLoss` 計算模型預測值與真實標籤之間的誤差。

Backward Pass (反向傳播): `loss.backward()`，透過 Autograd 系統，計算 Loss 對模型中每一個可學習參數的梯度。

Update Weights (更新權重): `optimizer.step()`，根據計算出的梯度與優化器，更新網路的權重參數。

Optimization & LR Scheduler

Optimizer: 使用 Adam 優化器，初始 Learning Rate 設為 0.001。幫助模型在訓練初期快速收斂。

Learning Rate Scheduler: 使用 `optim.lr_scheduler.StepLR(step_size=10, gamma=0.1)`。在第 10 個與第 20 個 Epoch 時，將學習率衰減為原本的 10%。讓模型在 Loss 的谷底進行更細緻的 Fine-tuning，避免訓練後期的震盪。

Validation Loop

設定 `model.eval()`，在每個 Epoch 訓練結束後進行測試集評估。

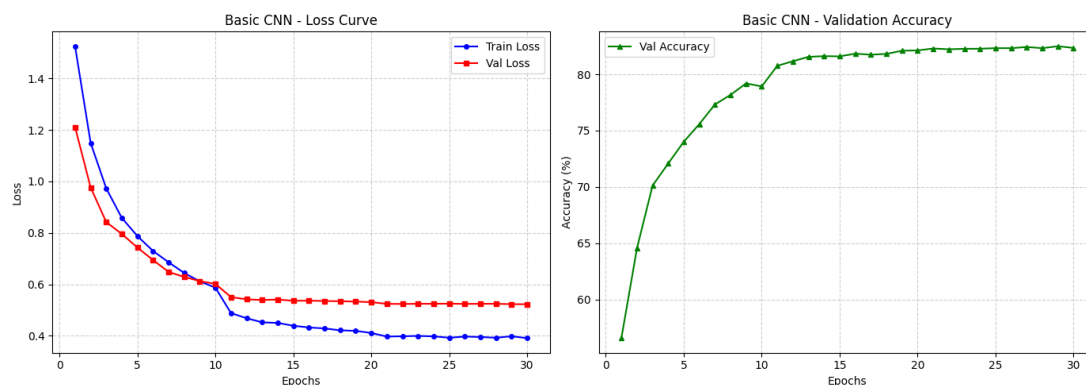
`model.eval()`：關閉 Dropout 並切換 Batch Normalization 層的行為，避免測試結果出現偏差。

`torch.no_grad()`：停止建立計算圖，減少推論時的 GPU 記憶體消耗並提升運算速度。

Experimental Analysis

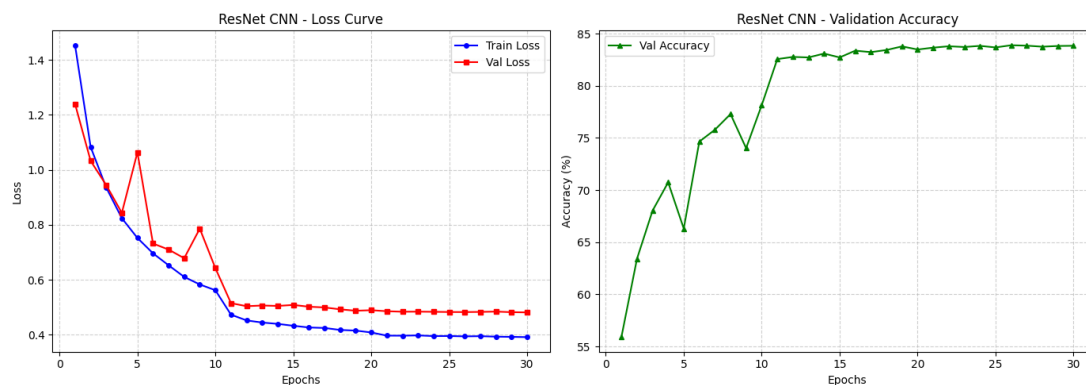
Basic CNN Accuracy: 82.34%
ResNet CNN Accuracy: 83.84%

基礎卷積神經網路 (Basic CNN) 訓練結果



Loss Curve 可以觀察到，Training Loss 與 Validation Loss 在前 10 個 Epochs 皆呈現穩定的下降趨勢，且兩者差距不大，因為 Data Augmentation 有效抑制了 Overfitting。在 Validation Accuracy 曲線中，準確率在初期爬升至約 78% 左右後開始放緩。

殘差卷積神經網路 (ResNet CNN) 訓練結果



在訓練前 10 個 Epochs 中，ResNet 的 Validation Loss 與 Accuracy 出現了較明顯的震盪(第 5 與第 9 Epoch 的尖峰)。因為 ResNet 架構中引入了 Batch Normalization，加上使用 Adam 優化器在較大初始學習率 (0.001) 下，對 mini-batch 的統計量較為敏感。雖然初期有震盪，但最終的收斂與泛化能力皆優於 Basic CNN。

Analysis and Reflection

Architecture & Parameter Efficiency

在架構設計上，Basic CNN 因為在卷積層後直接使用 Flatten 並接上龐大的全連接層 (FC Layer, 12888 $\rightarrow$ 512)，導致總參數高達 4.29M，且模型大小達到 16.38 MB。

反觀 ResNet CNN，使用了 Global Average Pooling，將空間特徵壓縮，把參數量縮減至 0.29M (僅 1.11 MB，參數減少約 93%)。參數減少的 ResNet，準確率卻上升 (83.84% > 82.34%)。證明 GAP 配合 BN 與殘差連接，能以高參數效率汲取泛化特徵。

FLOPs $\neq$ Latency

計算量分佈： Basic CNN 參數雖多，但計算量只有 0.014 GFLOPs；ResNet 參數少，但計算量反而較高 (0.030 GFLOPs)。這是因為講義所述，Conv layers dominate computation (~99.5%)，FC 層只是簡單的矩陣相乘，而 ResNet 較深的卷積層貢獻了更多的乘加運算 (MACs)。

推論記憶體與延遲： ResNet 的模型檔案只有 1.11 MB，但在推論時的峰值記憶體 (Peak Mem: 51.08 MB) 與延遲 (Latency: 0.83 ms) 卻都高於 Basic CNN (24.86 MB / 0.41 ms)。這是因為 ResNet 的 Skip Connection 必須將歷史的 Feature Maps 保留在 GPU VRAM 中等待後續相加，大幅增加了 Memory Footprint。模型小不等於跑得快，也不等於省記憶體。

Conclusion & Reflection

對比 Workshop A 中使用 NumPy 徒手推導 Forward/Backward Propagation，這次使用 PyTorch 的 nn.Module 與 Autograd 機制，讓我體會到了現代深度學習框架的強大與便捷。

在資源豐富的 GPU 上，我們或許可以隨意堆疊層數，但後續課程將面臨嵌入式設備的硬體限制，我非常期待能運用 Quantization 與 Pruning 技術，進一步消除卷積層的運算瓶頸，將模型真正推向 Edge Deployment 的階段。

Code Appendix

<https://github.com/Wzx0110/Building-a-CNN-on-CIFAR-10-with-PyTorch.git>